Verified Optimization, Export and Evaluation of a Translation from Real-Time Requirements to Timed Büchi Automata

Elie Fares[a,*], Jean-Paul Bodeveix[b], Azar Salami[a], Hussain Al-Aqrabi[a]

[a] *Higher Colleges of Technology, Sharjah, United Arab Emirates*
[b] *IRIT, Université de Toulouse, Toulouse, France*

Abstract

MTL requirements are checked by translating their negation into a timed Büchi automaton (TBA) that UPPAAL or TChecker composes with the system. We present an MTL-to-TBA chain whose optimization and export stages are verified in Rocq: the passes preserve the timed-word language of every TBA of the formula-indexed clock interface, for any number of rounds. Two further passes, removal of dominated transitions and reuse of clocks, run an untrusted analysis whose certificate a verified checker validates, so a wrong certificate costs optimization but not correctness. Two verified policies eliminate disjunctive location invariants; the all-copy policy yields no diagonal guards, so TChecker's liveness algorithms accept it. An end-to-end theorem adds a verified MTL translation; for Spot, which the tool runs, it needs a stated hypothesis and a successful initial-value check. Against CASAAL, ours have the same size on the negated requirements both tools handle and TChecker stores comparable states. On a constructed workload CASAAL's automaton costs TChecker 1,736,096 states against 114 for ours, and verified pruning after splitting removes the gap. Clock reuse removes 47 of 201 clocks and makes TChecker up to 28% faster on constructed products. Generation is slower than CASAAL's on deep nests. Of 22 seeded defects, 18 change the language and a random-word test misses five; Rocq rejects all 22.

*Corresponding author

Email addresses: efares@hct.ac.ae (Elie Fares), bodeveix@irit.fr (Jean-Paul Bodeveix), asalami@hct.ac.ae (Azar Salami), halaqrabi@hct.ac.ae (Hussain Al-Aqrabi)

Keywords: timed automata, Metric Temporal Logic, verified optimization, Rocq, UPPAAL, model checking of real-time systems

1. Introduction

Safety and liveness requirements of real-time systems are commonly written in Metric Temporal Logic (MTL) and checked by translating the negated requirement into a timed Büchi automaton (TBA), which a timed model checker such as UPPAAL [1] or TChecker [2] composes with the system model. The size and shape of that automaton matter: clocks multiply the symbolic state space the checker has to explore; repeated bounds, dead resets and redundant transitions enlarge the automaton without changing its language; and UPPAAL accepts only conjunctive guards and conjunctive upper-bound invariants, whereas merging transitions naturally produces disjunctions.

Optimizing a TBA is risky. A pass that is wrong in a rare case silently changes the set of timed words the property accepts, and a model checker then reports a verdict about a different requirement. The passes also interact: propagating a lower bound can make a transition contradictory, and splitting a disjunctive invariant into copies by a single entry choice is safe only if the chosen copy admits at least as much time as every other copy, which the all-copy policy of Section 5 avoids by entering every copy. A random-word test can miss defects of this kind (Section 9.6).

This paper presents a post-processing pipeline for TBAs whose every pass is mechanically verified in Rocq, together with its use in a translator from MTL. The *optimization* stage synthesizes upper-bound location invariants, propagates them into guards and entry bounds, removes contradictory transitions and dead resets, merges clocks that are always reset together, normalizes guards, and merges equivalent transitions and locations. Two further passes act on a global analysis whose result is checked and not trusted: the removal of transitions that another transition dominates, and the reuse of clocks. The *export* stage merges transitions by disjoining their guards and eliminates disjunctive invariants by creating copies with conjunctive invariants and clock-difference entry conditions, so that guards and invariants are conjunctive. We prove that each pass preserves the timed-word language of every input TBA, that the optimizer does so for every number of rounds, that the checked passes do so for every certificate, and that the output of export has the conjunctive shape UPPAAL needs.

The passes are stated for the TBAs of the formula-indexed clock interface, not only for those that our translator generates, and work together: normalizing guards makes dead resets visible, removing them makes more clocks synchronous, and merging those lets transitions and locations merge, so the optimizer is iterated. For MTL requirements we compose the passes with a translation whose correctness was established separately in Rocq [3], factored through clocked Linear Temporal Logic and proved correct for a verified propositional Büchi backend. Section 7 states the resulting theorem, which has no hypothesis on the backend when the verified backend is used; for Spot, which the tool runs, conformance to the backend contract is a hypothesis. This paper does not repeat the translation proofs; Section 2 fixes the definitions the passes are stated over. The result is a *modular, mechanized chain* in which the propositional backend is a parameter (Section 7.1).

The verified passes are extracted to OCaml and run inside the translator `mtl2tba`, which calls Spot [4] for the propositional automaton and writes UPPAAL XML. We ask five questions of the pipeline.

RQ1 How does the whole translation compare with an independent translator, CASAAL [5], in size, generation time and the states a timed model checker stores?

RQ2 What does the export cost, which export policy can the checkers read, and where does optimization fail to help?

RQ3 Which Spot setting should a user choose, and what does the call to Spot cost on deeply nested requirements?

RQ4 What does verification add: which plausible defects does testing miss that the proofs reject?

RQ5 What do the certificate-checked passes change, for the automata, for the model checker, and for the gap to CASAAL on the constructed workload?

The contributions are:

- a modular chain from MTL to the exported timed automaton whose stages are stated against interfaces, so that the propositional backend (Spot, or any translator that meets the contract) can be replaced without changing the optimization and export theorems; Spot, the readers of its output, the XML printer and UPPAAL lie outside the proofs;
- a Rocq development of language-preserving TBA transformations (invariant synthesis and propagation, clock and location simplification, transition merging, disjunctive-invariant elimination), with preservation theorems for every TBA of the formula-indexed clock interface and every number of op-

timization rounds;

- two certificate-checked passes, the removal of dominated transitions (after a verified splitting of transitions by cells of letters) and the reuse of clocks by liveness: an unverified OCaml procedure proposes a certificate, a Rocq-verified checker accepts or rejects it, and the language is preserved for every certificate (Section 4);
- two verified policies for eliminating disjunctive location invariants, the maximal-delay policy (which may introduce clock-difference constraints) and the all-copy policy (which introduces none and is the one that TChecker's liveness algorithms accept), a verified export with conjunctive guards and invariants (Sections 5 and 9.4), and an end-to-end theorem that composes the passes with a verified translation (Section 7), without hypotheses on the propositional backend when the verified backend is used (the shipped tool runs Spot instead);
- an evaluation against CASAAL in output size, generation time, TChecker stored states and UPPAAL verdicts, a differential test of both tools against an independent MTL evaluator (122 inputs, 6,160,000 word checks, no disagreement), and the passes on 600 independently generated TBAs; on the constructed workload where CASAAL's automaton costs TChecker 1,736,096 states against 114 for ours, the verified pruning removes the gap, and clock reuse removes 47 of 201 clocks;
- a seeded-defect study of 22 defects of our choosing in the extracted code: a random-word test misses five of the 18 that change the language, and the Rocq development rejects the corresponding mistakes, which measures the sensitivity of the proofs to edits and not defect detection; and
- negative results: no measurable change of what TChecker explores from the optimizer or from pruning of our own automata, weak pruning on generic graphs, and slower generation than CASAAL on deeply nested requirements.

2. Timed Automata and the Preservation Contract

2.1. Timed words and TBAs

A timed word is an infinite sequence $w = (a_i, t_i)_{i \in \mathbb{N}}$, where a_i is an event from a finite alphabet Σ, timestamps satisfy $t_i \geq 0$ and are nondecreasing, and time diverges: $\lim_{i \to \infty} t_i = +\infty$. For each position i, let $\delta_i = t_{i+1} - t_i$ be the time from event i to the next event. A clock valuation maps a finite

clock set X to $\mathbb{R}_{\geq 0}$. Guards of a TBA are finite conjunctions of event literals and unary clock comparisons $x \bowtie c$ for $\bowtie \in \{<, \leq, =, \geq, >\}$; transitions also carry a set of clocks to reset. The exported automaton (called a *DTA* below, after its Rocq type) is a TBA whose guards may in addition contain diagonal comparisons $x - y \bowtie c$ (Section 5), and whose locations carry conjunctive upper-bound invariants. In the *formula-indexed clock interface* used by the Rocq development, the clock set is the set of timed subformulas of a fixed formula, which serve only as clock names; an automaton with k clocks is represented by choosing a formula with at least k timed subformulas.

A TBA is a tuple $\mathcal{A} = (L, \ell_0, X, E, Acc)$, where L is a finite location set, ℓ_0 is the initial location, X is the clock set, E is a finite set of transitions $(\ell, \alpha, g, Z, \ell')$, α is an event condition, g is a clock guard, $Z \subseteq X$ is a reset set, and $Acc \subseteq L$ is the set of Büchi-accepting locations. The input automata of the verified optimization carry no location invariants; synthesis adds them as a separate annotation. Exported invariants are conjunctions of upper bounds.

The Rocq development interprets transitions over an extended timed word that records clock values and reset decisions. Here v_i is the clock valuation at event i, and Z_i records the clocks reset by the transition on that event. Since a reset occurs at t_i, the next-event valuation includes the elapsed interval δ_i:

$$v_{i+1}(x) = \begin{cases} \delta_i, & x \in Z_i, \\ v_i(x) + \delta_i, & x \notin Z_i. \end{cases}$$

A run chooses one enabled transition per position and visits accepting locations infinitely often. The accepted language existentially quantifies over clock-consistent extensions of the base timed word, including the initial clock values. This is the acceptance notion of the preservation theorems; conventional initialization at zero is a separate deployment concern, bridged by the verified check `init_free` (Section 8) for the reading in which all clocks start at 0, as in UPPAAL; the check succeeds when no clock can be read before it is reset from the initial location.

$\mathrm{MTL}_{0,\infty}$ is metric temporal logic with the pointwise semantics over timed words, whose timed operators carry intervals with lower bound 0 or upper bound ∞; $\mathsf{msat}(w, i, f)$ states that w satisfies f at position i, and the requirement is satisfied by w when $\mathsf{msat}(w, 0, f)$ holds. The definitions are those of [3].

For an annotated automaton, an invariant must hold throughout the delay

spent at its location. Since synthesized invariants are conjunctions of non-strict upper bounds and clocks are nondecreasing during a delay, holding at the event at the end of the delay is equivalent to holding throughout that stay. The proof formalizes this trace-model equivalence and uses it to show that the annotations and strengthening operations preserve timed words.

2.2. What export must preserve

The export must retain the same transition alphabet, Büchi marking, clock meaning, and timed-word language. It may change the number of locations, transitions, and clocks, and may introduce difference constraints in transition guards to choose among invariant copies. Such choices must not create or delete an accepted timed word. The target has conjunctive guards and location invariants; a later symbolic representation groups transitions with the same source, target and resets under Boolean labels.

The preservation theorems hold for every TBA of the formula-indexed clock interface, whether or not the MTL translator generated it; Spot's correctness and the soundness of the front end are outside them.

3. Verified Optimization

After invariant synthesis, the optimization pipeline alternates timing propagation with simplification. Synthesized upper bounds, propagated to incoming guards, can make constraints jointly inconsistent; forward propagation supplies lower-bound information. The pipeline then removes the transitions, resets, clocks and duplicates that no longer affect the language.

The proofs of the passes are mechanized in Rocq (Figure 1 shows the invariant computation); the development of this paper is 6,105 lines with 251 theorems and lemmas for the invariants, the optimization, the export and the symbolic grouping, and 1,535 lines with 50 for the checked passes of Section 4, on top of the translation of [3]. Each pass is a total function on TBAs with a preservation theorem listed in Supplement Table 13; the reasoning is the same for all: a run of the output is mapped to a run of the input (or conversely) location by location, and the changed component (a guard, an invariant, a reset set, a clock or a location) is shown to be implied or equivalent at every event. In brief:

- *Invariant synthesis* adds, at each location, the upper bound that the outgoing guards require; *backward propagation* strengthens incoming guards

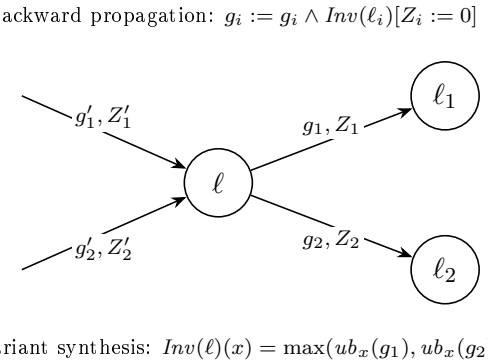


Figure 1: Invariant computation. Successor invariants strengthen outgoing guards; upper bounds of the resulting guards resynthesize the source invariant. Reset sets determine which target-clock constraints become constant.

with the target invariant (reset clocks become constants); *forward propagation* adds the lower bounds implied by the entry bounds.

- *Contradictory transitions* (unsatisfiable guards or labels) and *unreachable outgoing transitions* are removed; *dead resets* (resets after which no path tests the clock before it is reset again) are removed, and *synchronously reset clocks* are merged (every transition resets both clocks or neither, and those leaving the initial location reset the clock and do not test it).
- *Normalization* drops tautological and subsumed atoms; *transition merging* fuses transitions only by subsumption and resolution, that is, when the union of two guards is again a conjunction (disjunctive unions are left to export); *location merging* fuses locations that have the same outgoing behavior.

The definitions, the invariant semantics on the trace model and the proofs are in Supplement Section 1.

3.1. Iterated optimization

In right-to-left function-composition order, one optimization round is

$$\begin{aligned}\mathsf{optimize_step} = \mathsf{merge_states} \circ \mathsf{merge_trans} \circ \mathsf{normalize} \circ \mathsf{merge_clocks} \\ \circ \mathsf{dead_resets} \circ \mathsf{normalize} \circ \mathsf{unreachable} \circ \mathsf{unsat_labels} \\ \circ \mathsf{contradictory} \circ \mathsf{forward} \circ \mathsf{propagate}.\end{aligned}$$

The order is chosen so that the first normalization removes the tautology $x \geq 0$ that forward propagation can add after a reset, before dead-reset analysis, which would otherwise treat the clock as live; we did not test other

orders. Each operation may enable later simplifications, so the driver repeats the round until the TBA is structurally unchanged or 50 rounds have run. The theorem is independent of this stopping rule and holds for every n.

Theorem 3.1 (Optimization)**.** *For every TBA $\mathcal{A}$ of the formula-indexed clock interface and every $n \in \mathbb{N}$,* $\mathsf{optimize}(n, \mathcal{A})$ *accepts exactly the timed words accepted by $\mathcal{A}$.*

Proof. Every constituent operation preserves acceptance, as listed in Supplement Table 13. Their composition preserves acceptance, and induction on n proves the statement for every number of rounds. The mechanized result is `optimize_accepts`. □

The theorem concerns the TBA component of the result; the extracted function also returns synthesized upper-bound invariants and entry bounds as annotations, which the export does not receive. The driver stops when successive automata are structurally equal or after 50 rounds; this policy is not a premise of `optimize_accepts`. On the 29 completed benchmark inputs at most six rounds change the automaton (R6), so the cap was never reached.

4. Certificate-Checked Passes

The passes of Section 3 are local rewrites, and their proofs follow the rewrite. Two further reductions need a global analysis that is hard to prove correct and easy to check: the removal of transitions that another transition dominates, and the reuse of clocks. For both, an unverified OCaml procedure proposes a *certificate*, a boolean checker extracted from Rocq tests local conditions on it, and the pass returns the automaton unchanged when the check fails. The Rocq theorem is that the language is preserved when the check succeeds, so it quantifies over every certificate, including wrong ones, and nothing is assumed about the procedure. The scheme is the one of the validation of register allocation in CompCert [6]. Both passes run after the optimization rounds and before export; the pruning is preceded by a verified splitting of transitions.

4.1. Removal of dominated transitions

Two transitions out of one location may overlap: whenever the second is enabled so is the first, and the first leads to a location that can do everything that the target of the second can do. The second is then redundant for

the language but not for a model checker, which explores both. Syntactic subsumption (Section 3) sees only pairs with the same target and resets; simulation-based reductions of Büchi automata [7] are untimed, and clock guards and resets must be related in a timed setting.

The certificate is a set K of transitions to keep, a relation $\sim$ on locations ($q_1 \sim q_2$: q_1 simulates q_2), and for each location q two sets of clocks $\mathit{live}(q)$ and $\mathit{lle}(q)$. The checker `prune_ok` tests that the initial location simulates itself; that $q_1 \sim q_2$ and q_2 accepting imply q_1 accepting; and that for $q_1 \sim q_2$, every transition t_2 out of q_2 and every cell c of letters (see below) on which its label holds, some *kept* transition t_1 out of q_1 has a label that holds on c, a guard implied by that of t_2 and made of clocks of $\mathit{live}(q_1)$ or of upper bounds on clocks of $\mathit{lle}(q_1)$, resets that agree with those of t_2 on the clocks live after the step, and a target that simulates that of t_2. The two runs thus keep equal values on live clocks and the dominating run has smaller or equal values on the clocks of lle, which suffices for upper-bound guards; the resets may differ elsewhere (the exact conditions are in Supplement Section 3.2).

Theorem 4.1 (Pruning)**.** *For every certificate producer and every TBA $\mathcal{A}$, the automaton returned by* `prune_pass` *accepts exactly the timed words accepted by $\mathcal{A}$.*

Idea. Every run of the pruned automaton is a run of $\mathcal{A}$. Conversely, a run of $\mathcal{A}$ is mapped position by position to a run of the pruned automaton: at each step the transition taken is replaced by the kept transition that dominates it, and an invariant states that the two locations are related by $\sim$ and that the valuations agree on the live clocks and are ordered on the clocks of lle. Acceptance is preserved because accepting positions map to accepting positions. The mechanized result is `prune_pass_accepts`; if the check fails, the pass is the identity. □

Splitting by cells of letters.. A transition whose label holds on several letters may be dominated on some of them only. `split_cells` replaces each transition by one copy for every *cell* of letters in its label, one cell for each action mentioned in a label of the automaton and one for all other actions, each copy labelled by the literals that characterize its cell. This makes the label test exact, and `split_cells_accepts` shows that the language is unchanged. Splitting alone enlarges the automaton; it pays when the pruning that follows removes the copies that other transitions dominate (Section 9.3).

4.2. Reuse of clocks by liveness

Translators introduce one clock for each timed subformula, and clocks that are never read at the same time can share one variable, which lowers the dimension of every zone. Liveness, interference graphs and colouring are the method of Saeedloei and Kluźniak for clock allocation in timed automata [8]; ours is a Rocq-checked version for the semantics of this chain. The certificate is an allocation $\alpha : \mathcal{C} \to \mathcal{C}$ and sets $\mathit{live}(q)$ that over-approximate the clocks read before being reset from q. The renamed automaton $\alpha(\mathcal{A})$ replaces every clock x by $\alpha(x)$ in guards and reset sets. The checker `alloc_ok` tests, for every transition t from q to q': **(G)** the clocks of its guard are in $\mathit{live}(q)$; **(F)** $\mathit{live}(q') \subseteq \mathit{reset}(t) \cup \mathit{live}(q)$; **(R)** if r is reset and $y \in \mathit{live}(q')$ with $\alpha(r) = \alpha(y)$ then $r = y$; and **(I)** α is injective on *live* of the initial location.

Theorem 4.2 (Clock reuse)**.** *If* (α, live) *passes* `alloc_ok` *for* $\mathcal{A}$*, then* $\alpha(\mathcal{A})$ *accepts exactly the timed words accepted by* $\mathcal{A}$*;* `alloc_pass` *applies* α *only after a successful check.*

Idea. Along a run, a class of α is reset when one of its members is, and every live clock has the value of its class (by (I) at the start, by (F) and (R) at each step); with (G) the guards of the renamed transition hold exactly when those of the original do, and the converse direction picks the first transition that maps to the one taken, so no choice axiom is used. The mechanized result is `try_allocate_accepts`; the conditions are discussed in Supplement Section 3.3. □

The certifier computes the least liveness sets by a fixpoint, builds the interference graph (an edge between a reset clock and each clock live at the target, and between the clocks live initially) and colours it with DSATUR [9]; it is heuristic and need not find the fewest clocks. The semantics is that of the rest of the chain, with existentially chosen initial clock values.

Composition.. The checked passes compose with the translation: for every pruning certifier, every allocation certifier, every n and every well-formed f, a timed word satisfies f iff the export of $\mathsf{alloc_pass}(\mathsf{prune_pass}(\mathsf{split_cells}(\mathsf{optimize}(n, \mathsf{compile}(f)))))$ accepts it (`MTL_to_exported_checked_correct`); the variant for the Spot-parametrized chain (`MTL_to_exported_checked_correct_with`) carries the same hypothesis on the propositional automaton as Theorem 7.1. Under UPPAAL's initialization the guarantee needs the check of Section 8 on the final automaton: if `init_free` succeeds on the export of the checked chain, then

$\mathsf{msat}(w, 0, f)$ holds iff the exported automaton accepts w with every clock initialized to 0 (`MTL_to_exported_checked_correct0_recur_weak_with`, and `MTL_to_exported_checked_correct0_with`), under the same Spot contract and for every certificate; the tool runs the check after pruning and clock reuse. The two passes, with the split and these statements, add 1,535 lines and 50 lemmas and theorems to the development; `Print Assumptions` lists only the standard library principles of Section 8.

5. Mechanically Verified Elimination of Disjunctive Invariants

Disjunctive invariants do not arise in the optimization of Section 3, whose synthesized invariants are conjunctions of upper bounds. They arise in export, where the upper bounds that the outgoing guards of a location impose form a disjunction of conjunctions (transitions that agree on source, target, label and resets are also merged by disjoining their guards). This section eliminates such disjunctions.

5.1. A worked example: selecting a longest admissible delay

Suppose the invariant synthesized at a location q is $I_x \vee I_y$, where $I_x \equiv x \leq 3$ and $I_y \equiv y \leq 4$. From the entry valuation $(x, y) = (0, 0)$, the original disjunction allows a stay of up to four time units: I_y remains true after I_x has expired. A single copy carrying only I_x would stop the run after three units and could lose a timed word whose next event occurs at time four. The copies therefore cannot be selected by an arbitrary disjunct.

The split creates q_x with invariant $x \leq 3$ and q_y with invariant $y \leq 4$. A copy is enabled on entry only when it admits a delay at least as long as every other copy. With the difference-constraint convention $u - v \leq c$, the entry predicates are

$$G_x \equiv x \leq 3 \ \wedge \ x - y \leq -1, \qquad G_y \equiv y \leq 4 \ \wedge \ y - x \leq 1.$$

At $(0, 0)$, G_y holds and G_x does not, so the four-unit branch is selected. At the tie valuation $(0, 1)$, both predicates hold and both copies admit the same maximal delay. Comparing residual times $3 - x$ and $4 - y$ requires the clock-difference constraints; separate unary guards cannot express their ordering.

The reset-adjustment step is visible when an incoming edge resets x but leaves y unchanged. Write y^- for the value of y just before the edge; immediately after the edge, $x = 0$ and $y = y^-$. Substituting this reset into the

two entry predicates gives

$$G_x[x := 0] \equiv y \geq 1, \qquad G_y[x := 0] \equiv y \leq 1.$$

Thus $y^- > 1$ admits only the x-copy, $y^- < 1$ only the y-copy, and the tie $y^- = 1$ admits both. The comparison of two clocks has become a unary comparison because one clock was reset. If the edge resets both clocks, a difference constraint becomes a constant truth test; if it resets neither, the diagonal comparison remains. The case in which only y is reset is analogous. Together these are the four reset cases handled by the verified guard-substitution function.

5.2. General construction and the two split policies

For a disjunction $I_1 \vee \cdots \vee I_r$ of upper-bound conjunctions, the construction creates one copy of the location per irredundant conjunction U_k, carrying U_k as its invariant. The *maximal-delay* policy admits a copy on entry only when its largest admissible delay is at least that of every other copy; this needs clock-difference (diagonal) entry guards, which are adjusted by the verified guard substitution for the reset cases of the example above. The *all-copy* policy lets every copy be entered and relies on the covering property proved inside the construction (`disj_inv_cover`); it is nondeterministic and introduces no diagonal constraint. Both preserve the timed-word language (`split_accepts`, `split_all_accepts`); the full definitions, the proofs of both and a measured comparison on synthetic automata are in Supplement Section 2.1. On two synthetic examples the measured trade-off is not uniform: with two clocks all-copy uses four more edges but fewer guard atoms and no diagonal atoms; with six branches both give 68 edges and all-copy has fewer atoms; Section 9.4 compares the two policies on the benchmark. The default remains maximal delay; all-copy is separately verified and used whenever TChecker's liveness algorithms must read the automaton.

6. Verified UPPAAL-Oriented Export

6.1. Generated continuation example

Supplement Figure 3 follows an automaton generated by the current tool from the reset-completion boundary through optimization and export. It does not repeat the upstream MTL translation. In this instance, two optimization rounds remove nine transitions; export adds three explicit transitions while producing the required conjunctive internal shape.

The export continues by splitting disjunctive guards into conjunctive transitions, normalizing, removing copies that are never entered, and merging the initial location when an equivalent copy exists. A final symbolic pass groups transitions with the same source, target, and resets and stores their labels as Boolean conditions for drawings.

6.2. Whole-export preservation and output shape

Theorem 6.1 (Export preservation)**.** *For every invariant-free TBA given to the verified export interface, the explicit export preserves its timed-word language. The resulting automaton has conjunctive transition guards and conjunctive location invariants.*

Proof. The explicit construction composes verified transition merging, invariant splitting, guard splitting, normalization, removal of copies never entered, and initial-location merging; its timed-word preservation result is `export_accepts`. The output shape is established by `export_guards_conjunctive` and `export_invariants_conjunctive`. A separate symbolic grouping pass preserves timed-word acceptance by `symbolic_accepts` (with the extended-word result `symbolic_ext_accepts`). □

For reuse with another producer or checker, the theorem's output contract is summarized in Supplement Table 11: the language is preserved, guards and invariants are conjunctive, and a third-party parser or conversion needs its own correspondence proof.

The generated files for the R2 input (the input, the optimized automaton and the export) are in `evaluation/generated_examples/R2/`, with their run metadata. This example shows how the optimizer and the export interact; the synthetic suite of Section 9.5 isolates pass effects on direct TBA records.

7. End-to-End Guarantee for MTL Requirements

The passes of Sections 3–6 are stated over arbitrary TBAs. For MTL requirements they are used behind a translation $\mathsf{compile_with}(A)$ that turns a propositional Büchi automaton A for the clocked-LTL encoding $\mathcal{T}(f)$ of an $\mathrm{MTL}_{0,\infty}$ formula f into a TBA. Its correctness, in the form we need, is the result of [3]: if for every action-consistent propositional word s, A accepts s exactly when s satisfies $\mathcal{T}(f)$ (the *backend contract*, which any A with the

language of $\mathcal{T}(f)$ meets) and f is well formed (nonnegative non-strict upper bounds, strictly positive bounds for the other timed operators), then

$$\mathsf{msat}(w, 0, f) \;\leftrightarrow\; \mathsf{TBA_accepts}(\mathsf{compile_with}(A), w)$$

for every timed word w with nondecreasing timestamps (the words of Section 2 are a subclass). The same work provides a propositional automaton $\mathsf{backend}(f, \mathcal{T}(f))$ for which the backend contract is proved in Rocq.

Our development imports these definitions and results, and instantiates the generic composition with that backend:

Theorem 7.1 (End-to-end correctness with the verified backend)**.** *For every $n \in \mathbb{N}$, every well-formed $MTL_{0,\infty}$ formula f and every timed word w,*

$$\mathsf{msat}(w, 0, f) \;\leftrightarrow\; \mathsf{DTA_accepts}\big(\mathsf{export}(\mathsf{optimize}(n, \mathsf{compile_with}(\mathsf{backend}(f, \mathcal{T}(f))))), w\big).$$

The same holds with the all-copy exporter of Section 5 in place of export*. Here the invariant-free TBA component of the result of* optimize *is the input of* export*; the invariants and entry bounds that the extracted function also returns are not passed to it.*

Proof. Let $C = \mathsf{compile_with}(\mathsf{backend}(f, \mathcal{T}(f)))$, $O_n = \mathsf{optimize}(n, C)$ and $D_n = \mathsf{export}(O_n)$. The translation result gives $\mathsf{msat}(w, 0, f) \leftrightarrow \mathsf{TBA_accepts}(C, w)$, the backend contract being discharged by the verified backend. Preservation by optimization (Theorem 3.1) gives $\mathsf{TBA_accepts}(C, w) \leftrightarrow \mathsf{TBA_accepts}(O_n, w)$, and preservation by export (Theorem 6.1) gives $\mathsf{TBA_accepts}(O_n, w) \leftrightarrow \mathsf{DTA_accepts}(D_n, w)$. The Rocq statements are `MTL_to_exported_correct_verified` and `MTL_to_exported_all_correct_verified`; `Print Assumptions` lists only the standard axioms of the Rocq library (classical logic, functional extensionality, constructive definite description, and the real-number axiom of the standard library). □

The general form `MTL_to_exported_correct_with` replaces the verified backend by any automaton A that satisfies the backend contract. The verified backend performs no simplification and is a proof witness only; it is not the one used by the extracted tool.

The theorem that the shipped tool instantiates is `MTL_to_exported_correct0_recur_weak_with`, the zero-initialized variant of the end-to-end theorem. Its statement is: for every n, every well-formed f,

every propositional automaton A for the rewritten formula $\mathsf{recur}(f)$ that accepts the action-consistent words satisfying the weakened clocked encoding of $\mathsf{recur}(f)$, and every timed word w, if `init_free` succeeds on $\mathsf{export}(\mathsf{optimize}(n, \mathsf{compile_with}(A)))$ then $\mathsf{msat}(w, 0, f)$ holds iff that automaton accepts w under the zero-initialized reading. Spot supplies A; whether its output satisfies the hypothesis on A is not proved, and is tested only by the translation validation and the differential checks of Section 9.6. The same statement holds for the complete chain with pruning and clock reuse, for every certificate, when `init_free` succeeds on the final exported automaton (`MTL_to_exported_checked_correct0_recur_weak_with`, Section 4). After a run, a user may conclude that the exported automaton accepts the requested language provided Spot's automaton meets the backend contract, `init_free` succeeded, and the trusted components listed in Section 8 behaved correctly.

The end-to-end theorem `MTL_to_exported_correct_with` concludes in the existential initial-clock semantics of Section 2 on the internal exported record. The two zero-initialized variants, `MTL_to_exported_correct0_with` and the `recur_weak` one above, add the hypothesis that `init_free` succeeds on the exported automaton and conclude under the zero-initialized reading. None covers the HOA/LBTT importer, the XML printer or parser, or UPPAAL itself (Section 8).

7.1. A modular chain: what can be replaced

Theorem 7.1 composes stages that are each stated against an interface: the clocked-LTL translation $\mathcal{T}$ (proved in [3]); a propositional backend, any Büchi automaton A that accepts the action-consistent propositional words satisfying $\mathcal{T}(f)$ (the contract of Section 7, a hypothesis of `MTL_to_exported_correct_with` and discharged by the verified, slow backend in `MTL_to_exported_correct_verified`); reset completion `compile_with` (proved in [3]); and the optimization, checked passes and export of this paper, which preserve the language of every TBA of the formula-indexed interface.

The backend is therefore a parameter: any translator whose output satisfies the contract can replace Spot without touching a proof. In this paper we changed Spot's level and options, the format in which we read its output, and replaced the list scans of the passes with tables with equality lemmas (Section 8.3), each time with no change to the statement of any theorem. A construction that produces a TBA directly from the formula, as CASAAL

does with the construction of Li et al. [10], would be composed with the same theorems and would remove the dependency on Spot; we have not written one, and no second backend has been plugged in, so the modularity is a property of the theorem statements and not an instantiated one. The design has a price: on deeply nested requirements the front end is the slow part of the chain, at 60 to 90% of CASAAL's whole translation time on seven levels in the one profile we made (Section 9.2.2); the cause may lie in Spot or in the formula of size $\Theta(3^k)$ that the verified translation hands to it.

8. Tool Architecture and Use in UPPAAL

8.1. Implementation and proof boundary

The OCaml module `optim.ml` is extracted from Rocq and contains the verified optimization and export functions. The integrated executable also contains handwritten input, iteration, and output code. These adapters and the XML/DOT printers are outside the transformation proofs. A converter from an external automaton format must establish its own correspondence with the Rocq TBA type. A clean Rocq 9.0.1 build, an independent `coqchk` run and an OCaml 5.4.0 rebuild are recorded in the artifact audit.

The project-wide axiom `LTL_TO_BUCHI_CORRECT` states the assumed correctness of the upstream LTL-to-Büchi backend and is used only by convenience theorems. The theorems `optimize_accepts`, `split_accepts`, `export_accepts`, `prune_pass_accepts`, `try_allocate_accepts` and `MTL_to_exported_correct_with` do not use it: `Print Assumptions` lists only standard classical Rocq library principles (including `ClassicalDedekindReals.sig_forall_dec`).

In the integrated workflow the translation of [3] produces the TBA that the optimizer receives; the driver then applies export to the optimized automaton.

8.2. Choosing the Spot level: automatic mode and manual override

The propositional automaton is produced by `ltl2tgba -B -<level> [-x <options>] -hoa` and read back by the tool (the reader is outside the proofs). The theorems hold for every automaton that meets the backend contract, so level and options affect running time strongly, the size of the result little, and correctness not at all. No level is best everywhere: level `low` is fast on conjunctions of bounded responses and slow on deeply nested bounded eventualities, and level `any` the reverse; Spot's simulation-based

reduction removed nothing on our inputs and takes more than half of Spot's time at level `any`, so we switch it off (`-x ba-simul=0`). The default *automatic mode* (`-spot-level auto`) runs Spot at level `low` and, after a budget of b seconds (`-spot-budget`, default 5), stops it and runs level `any` with `ba-simul=0` on the formula simplified by the proved simplification `ltl_simp_correct`. The fallback costs at most b seconds, and since it depends on a wall-clock measurement the level, and with it the run time, can differ between runs and machines; benchmarking should fix the level with `-spot-level`, `-simp` and `-spot-x`. The measurements are in Supplement Section 11.

8.3. Verified speedups, input contract and artifact layout

The passes were first written for the proofs, with list scans that dominated the time on the larger automata. We replaced them by per-location tables, a cheaper merge test, a duplicate-free initial-value search and a single computation of the formula's clock list. Each replacement carries a proof of equality with the definition it replaces, so every theorem applies to the definitions from which the fast code is extracted (byte-identical outputs on 92 runs form a regression test, not a proof). On the seven-level negated nested requirement N7 (129 locations and 10,930 transitions before optimization, Spot's own time not included), reset completion falls from 6.1 to 0.7 s, optimization from 15.2 to 2.2 s and the initial-value check from 15.8 to 1.1 s; on the positive form of R6, optimization falls from 19.6 to 0.29 s and the default export from 9.6 to 0.08 s (single runs of single stages, not the wall-clock times of complete runs of Section 9.4). Two changes concern the call to Spot and are outside the proofs: Spot's simulation-based reduction is switched off at level `any`, which cut Spot's time on the nested requirements by up to 2.8 in single runs, and the tool reads Spot's HOA output instead of LBTT. An external automaton enters the verified stages through the input contract (location acceptance, formula-indexed clocks, no location invariants), and `tba_opt` reads such an automaton from a text file. The rewritings, the input contract and the artifact layout are in Supplement Sections 4.1, 4.2 and 4.3.

8.4. Using exported automata in UPPAAL

The language-preservation theorem concerns infinite timed words with one event at each position, nondecreasing timestamps, divergent time, and the development's clock initialization semantics. A UPPAAL composition must preserve the assumptions needed to interpret its traces in that model:

(C1) *Events and time.* The system must synchronize each observable event with the property automaton, prevent zero-delay consecutive events (for example, reset a system clock z on each emission and require $z > 0$ on the next), and admit time-divergent executions; reachability alone does not rule out a Zeno continuation.

(C2) *Initial clocks.* UPPAAL initializes clocks to zero, so the zero-initialized conclusion requires the `init_free` check to succeed on the exported automaton.

(C3) *Acceptance.* UPPAAL's reachability interface does not check Büchi acceptance. For safety, use a violation automaton with an accepting sink that loops on every event; liveness requires a Büchi emptiness checker restricted to time-divergent runs, for which the artifact uses TChecker.

(C4) *Time constraints in products.* Location invariants can block time in a product, so an exported property automaton is not a passive observer unless its effect is justified; check deadlock and timelock conditions on the system.

(C5) *Numbers.* The tool accepts integer bounds in $[0, 2^{30}]$, while the theorems are over real-valued clocks.

The generated XML has a `Property` template for the automaton and an `Env` template for standalone exploration; in a system check the system replaces `Env` and synchronizes its observable events with `Property`, and the event `other` denotes any event outside the alphabet of the formula. Accepting locations are named `A`n. For a safety requirement, under (C1)–(C3) the query `E<> P.A` on a violation automaton composed with such a system detects a violating behavior; this query-level argument is not proved by the Rocq theorems.

9. Evaluation

9.1. Questions, evidence, and setup

This section evaluates the whole translation from an MTL requirement to the automaton given to a timed model checker, and the passes inside it. Section 9.2 compares the translation with CASAAL in size, generation time and stored states (RQ1); Section 9.3 evaluates the checked passes (RQ5); Section 9.4 the export policies, and Section 8.2 the Spot settings (RQ2, RQ3); Section 9.6 checks the exported automata against the semantics and asks

what verification adds (RQ4). Output shape is a Rocq theorem; verdicts, sizes and times are experimental evidence.

We retain the 31-attempt F/R/N benchmark: F is the group of single requirement patterns from the literature, Rn the conjunction of n bounded responses and Nn the nest of bounded eventualities of depth n; "positive form" is the requirement as written and "negated" the form given to the model checker. Of the 31 attempts, 29 completed the upstream stage and supplied an optimizer input (R7 and R8 timed out, and with the implementation of Section 8.3 their positive forms complete in 14 s and 90 s). All experiments ran on Ubuntu 26.04/WSL 2, an Intel Core i7-8650U at 1.9 GHz with 11.7 GB of memory, OCaml 5.4.0, Rocq 9.0.1 and Spot 2.16, with a 300 s limit unless stated. Generation times are medians of five runs, TChecker state counts are exact, UPPAAL times are medians of three runs, and other protocols are stated where used. The generated-group numbers were measured with Spot level `small` and the implementation before the speedups, and numbers from different builds are never added together. Across the 29 completed inputs the reachable transitions fall from 20,204 after reset completion to 1,500 after optimization (R6 accounts for 14,985 of the 18,704 removed), and export grows the explicit transitions to 3,209, a representation cost (Supplement Sections 6 and 9).

9.2. Comparison with CASAAL: the whole translation

CASAAL [5] implements the $\mathrm{MTL}_{0,\infty}$ construction of Li et al. [10]. It builds its own automata, so it is a reference for the whole translation, not for the optimization passes alone. We compare the size of the automaton, the generation time and the states a timed model checker stores. CASAAL ran with mutually exclusive events, which matches our one-event-per-position words, and its transition acceptance is converted to location acceptance, the form TChecker reads (Section 9.2.1). *Auto* is our default (Spot at level `low` for at most 5 s, then level `any` with Spot's simulation reductions off, Section 8.2); *manual* is the faster of the explicit settings `-simp -spot-level any -spot-x ba-simul=0` and `-spot-level low`. Both give automata of the same size on every pattern compared and differ in time only.

9.2.1. Size

The conversion of CASAAL's transition acceptance to location acceptance is done by a small importer outside the proofs, which reads CASAAL's DOT output and, when CASAAL prints no acceptance mark, treats a state as

accepting unless its label contains an unbounded eventuality. It does not convert the negated F14, which has three acceptance sets (the positive F14 converts). Every size and every state count of CASAAL's automata below depends on this conversion, which is not proved and is checked only indirectly, by the mutual language inclusion with ours that TChecker establishes on the small patterns and by the sampled words of Section 9.6; another encoding of the acceptance could give different counts.

Requirements as written (positive form). On the 27 patterns that CASAAL encodes exactly and both tools completed, ours has 222 locations and 976 transitions (Spot level `small`, which produced the archived runs) against 225 and 983 for CASAAL after conversion; ours is smaller on four patterns (F8, F11, F12, F14), equal on 21 and larger on F5 and F6 (25 transitions against 22).

Negated requirements (what the model checker receives). Table 1 and Supplement Table 14 give the sizes for R2–R8 and N5–N8: ours equals CASAAL's on every one, and the optimizer's output has exactly the size of CASAAL's on the nested family (N5: 461 transitions, N6: 1,541, N7: 5,105). On the 20 patterns whose products TChecker completes (the 18 of Table 2, F14, and N4), ours is smaller on F5 and F6 (3 locations and 5 transitions against 4 and 7) and on F8 (2 and 3 against 4 and 7), and equal on the others.

Isomorphism. Equal counts do not make equal automata: up to renaming, ours is isomorphic to CASAAL's imported automaton on 11 of the 27 negated requirements whose automaton the importer converts (F1, F2, F3b, R1–R8) and not on the other 16, and the optimizer leaves CASAAL's automaton unchanged on all 27, so it is a fixed point of our passes. TChecker finds the languages mutually included on F1–F12, R1 and N1–N4 (`evaluation/tck_equivalence.tsv`); on the larger products equivalence rests on the sampled words of Section 9.6.

9.2.2. Generation time

Table 1 reports the wall-clock time of the whole translation of the negated requirement with one driver for both tools (`evaluation/baselines/time_same_driver.py`). Ours runs under WSL 2 and CASAAL as a Windows executable, so the comparison carries the overhead of the virtual machine for our tool; a CASAAL run on the smallest patterns costs 0.07 to 0.10 s, which bounds the process start. On the bounded-response family and the small patterns both tools finish in under 0.3 s. CASAAL is faster from five nesting

Table 1: Negated requirements: CASAAL against `mtl2tba`. Size is reachable locations/transitions of the automaton given to the model checker (ours: after optimization). Times are wall-clock medians of five runs for the whole translation (for ours: generation, optimization and export, in automatic mode), both tools launched by one driver on the same machine, CASAAL as the native Windows executable and ours under WSL 2. N8 is a single run of each tool. The rows are a selection; the other patterns are in the supplement. Supplement Table 14 gives earlier runs, with the manual setting.

Pattern	CASAAL (loc./trans.)	Ours (loc./trans.)	CASAAL (s)	Ours, auto (s)
R2	4/8	4/8	0.07	0.05
R3	5/11	5/11	0.10	0.04
R5	7/17	7/17	0.15	0.06
R7	9/23	9/23	0.21	0.07
R8	10/26	10/26	0.27	0.05
N5	33/461	33/461	0.44	0.85
N6	65/1,541	65/1,541	2.56	6.65
N7	129/5,105	129/5,105	16.9	30.7
N8	257/16,769	257/16,769	74.3	307

levels: 0.44 s against 0.85 s on N5, 2.6 s against 6.7 s on N6 and 17 s against 31 s on N7; on N8, one run of each took 74 s and 307 s. With the manual setting, in the earlier runs of Supplement Table 14, ours takes 0.2, 1.3 and 15.1 s on five, six and seven levels against 0.3, 1.9 and 11.3 s; the automatic mode is slower because Spot first runs for up to 5 s at level `low`, and the two sets of times come from different runs and are only indicative.

On N7 about 10 of the 15 s of the manual setting are Spot's: a bounded eventuality repeats the translation of its operand three times, so the clocked formula has size $\Theta(3^k)$ in the nesting depth k. The front end, Spot together with the formula that the verified translation hands to it, and not the verified stages, appears to be the slow part (one profile; Supplement Section 9.1).

9.2.3. Cost for a timed model checker

We compare the states that TChecker stores (`tck-liveness -a couvscc`) on the product of the negated requirement with a reference system, the automaton of the requirement itself, which accepts exactly the words that the negation rejects. The product is empty and TChecker explores it completely, so the count does not depend on the exploration order. The reference is exported with the all-copy policy, because TChecker's liveness algorithms reject diagonal constraints ("unsupported diagonal constraints") and the maximal-delay export of the positive form of R2 and the

Table 2: TChecker stored states on the product of the negated requirement with the reference system, summed over the 18 products on which every variant completed (F1–F12 with F3b in place of F3, R1–R3, N1–N3; the 20 completed products without F14 and N4), and over three subsets of these products. "Optimizer disabled" is `-n 0`; "invariants removed" deletes every location invariant from the automaton given to TChecker. The per-product counts are in Supplement Table 15.

Automaton given to TChecker	all 18 products	F5–F8	F10–F12	R3
ours (default)	204,564	771	2,943	168,484
ours, optimizer disabled	204,564	771	2,943	168,484
ours, invariants removed	204,749	771	3,128	168,484
CASAAL, location acceptance	204,819	1,026	2,943	168,484
same, invariants removed	205,004	1,026	3,128	168,484

larger bounded-response patterns contains them (Section 9.4). TChecker labels locations and not transitions, so CASAAL's automata are given to it in location-acceptance form (Section 9.2.1) and exported by our tool like ours.

Over the 18 products TChecker stores 204,564 states on our automata and 204,819 on CASAAL's (Table 2). The counts are equal on 14 of the 18 products. On F5–F8 TChecker stores 771 states on ours and 1,026 on CASAAL's; CASAAL's automata are never smaller. On F7 the two automata have the same size; they differ by a guard $x_0 \leq 5$ on one loop (Section 9.2.1). Per product, the ratio of CASAAL's count to ours ranges from 1.0 to 6.0 (F8: 9 for ours against 54). The total is dominated by R3, on which both tools store 168,484 states. On R4–R7 and N5–N6 TChecker did not finish within 300 s for either tool (on R4 it had used 1.7 GB), and for R8, N7 and N8 the reference system or the automaton could not be generated within the limit, so nothing is claimed for those.

9.2.4. A workload on which the automata differ

The comparison above gives almost equal costs. The positive-form workload of Section 9.4 shows a large difference, but its first version exercised one response only. We therefore used a system that makes and answers in time the requests of the responses $2, \ldots, n$ and then makes the request of the first response and never answers it, so that all n responses of Rn are exercised and the product is empty (Supplement Table 17).

Table 3 supports four observations.

1. The difference grows with the number of responses exercised. Our count rises by 22 states for each added response (48, 70, 92, 114, 136);

Table 3: TChecker stored states on the positive form of the bounded response Rn composed with a system that exercises all n responses and violates the requirement (empty product, all-copy export, complete exploration). CASAAL's automaton is converted to location acceptance and exported by our tool like ours; the ratio is that of the stored-state counts. The last two columns apply the checked passes of Section 4 to CASAAL's automaton before the export: pruning of dominated transitions, and the same after splitting by cells of letters (Section 9.3). R5 did not finish for CASAAL's automaton, or after pruning alone, within the 180 s limit of that run (shorter than the 300 s used elsewhere), which is what "–" marks; R1 was not run with the passes.

Requirement	ours	CASAAL	CASAAL / ours	CASAAL pruned	CASAAL split, pruned
R1	48	48	1	–	–
R2	70	890	13	781	70
R3	92	47,008	511	34,483	92
R4	114	1,736,096	15,229	1,202,691	114
R5	136	–	–	–	136

CASAAL's rises by factors of 19, 53 and 37 (48, 890, 47,008, 1,736,096). Five points do not establish a complexity class.

2. The difference is not removed by the verified optimizer or by the export, and it is present in the zone graph of the converted CASAAL automaton; with the optimizer disabled our counts are the same, and passing CASAAL's automaton through our export changes its counts by at most 14% (2,000,526 to 1,736,096 on R4). The two automata have the same number of locations and clocks (2^n and n), but ours has about 1.2 edges for each location and event and CASAAL's has 2.5 on R2 and 8.7 on R5, and TChecker's zone graph has at most 17 zones for any location vector of ours against up to 7,444 for CASAAL's (Supplement Tables 18 and 29). Resetting dead clocks leaves the counts unchanged, and the gap shrinks to 2.8 to 14 times under inclusion subsumption. The conversion of CASAAL's acceptance is not the cause: for these requirements CASAAL prints no acceptance mark, and the importer keeps the printed locations and edges (Supplement Section 3.5).
3. The difference is specific to this workload: against a system that satisfies the requirement, CASAAL's automaton stores 1.2 to 1.8 times as many states as ours on R1–R3, and exactly as many through our export.
4. The extra states are explored through dominated transitions. After the verified splitting by cells of letters and the removal of dominated transitions (Section 4), TChecker stores exactly our counts on CASAAL's automaton for R2–R5 (70, 92, 114, 136), and R5 finishes (Section 9.3).

The workload is constructed and covers the bounded-response family only; it is no general speedup. It shows that two translators whose negated automata have the same size can give positive forms that a model checker explores at very different cost.

9.2.5. Sensitivity of the counts and non-empty products

With the optimizer disabled (`-n 0`) TChecker stores the same number of states on each of the 20 products, although the optimizer reduces the automaton (N7: 10,930 to 5,105 transitions, the size of CASAAL's automaton); on every CASAAL automaton that the importer converts, the optimizer changes nothing. Composing the negated automata with a free system and, for R1–R3, with a response system gives non-empty and empty products on which ours and CASAAL's variants agree in every verdict (138 and 84 verdicts, under both export policies; Supplement Section 11.2).

9.3. Certificate-checked passes (RQ5)

Pruning on the translator's own automata. We ran `mtl2tba -prune` (without splitting) on the 29 benchmark configurations and the 100 random formulas, in both polarities (`evaluation/prune_survey.tsv`); the checker accepted every certificate. On the requirements as written, the explicit transitions of the exported automaton fall from 1,504 to 1,025 on the benchmark (15 of 29 automata reduced; R6 640 to 448) and from 2,384 to 1,979 on the random formulas (21 of 100 reduced, none increased). On the negated requirements the effect is small (913 to 912 and 1,203 to 1,142). TChecker stores the same states with and without pruning on the reference products of R2–R4 (70, 92, 114): like the optimizer, pruning shrinks our own automata without changing what the checker explores.

Pruning on CASAAL's automaton. Table 3 adds two columns to the workload of Section 9.2.4: CASAAL's automaton after pruning, and after the verified splitting of transitions by cells of letters followed by pruning. Pruning alone removes a part of the extra states (890 to 781 on R2, 47,008 to 34,483 on R3, 1,736,096 to 1,202,691 on R4). Splitting first, so that overlapping labels can be compared, brings the count to exactly ours on R2–R5 (70, 92, 114, 136), and R5, which did not finish for CASAAL's automaton, finishes in 1.5 ms. On this workload the extra states of CASAAL's automaton are therefore explored through transitions that other transitions dominate and that the checked pass removes. This does not come from our conversion of CASAAL's acceptance (Section 9.2.1): for R2–R5 CASAAL

prints no acceptance mark, so the importer keeps its 2^n locations and marks every state accepting, and the transitions that TChecker receives are those that its printed labels denote, 12 to 13% of which repeat another transition of the same edge and the others of which differ in target, guard or resets (Supplement Section 3.5). What is ours is the reading of the labels as mutually exclusive letters and their expansion into one transition per disjunct.

Pruning on independently generated TBAs. We ran both variants on the 600 automata of Section 9.5: every certificate was accepted and the bounded word test finds no language difference. Pruning is not generally strong: it removes 24 of 3,054 transitions on 23 of the 300 graphs and nothing on the 300 monitors, and splitting enlarges the automata (3,054 to 5,316 and 1,930 to 3,078). It pays where a translation guesses which obligations are pending, as on CASAAL's automata.

Clock reuse. The pass removes 47 of the 201 clocks of the optimized negated automata (10 of the 29 benchmark automata and 16 of the 100 random formulas are reduced; 15 of 205 clocks of the forms as written), the checker accepts every certificate, and the numbers of locations and transitions never change since the pass only renames (`evaluation/alloc_survey.tsv`). To test the checker, we sent the clocks of 69 negated automata to random clocks (552 allocations; `evaluation/alloc_fuzz.tsv`): it accepted 343 and rejected 209, and applied without the check, 112 of the rejected allocations changed the language on 200 random words against an independent evaluator, and none of the accepted ones did. For the model checker, we composed the negated automata of the bounded-response families $\mathrm{B}(n, k)$ (n responses with bound k, with n timers that reuse one clock) and of three benchmark patterns with a system that satisfies the requirement, and ran TChecker on 15 pairs, five interleaved runs each, pinned to one core. Verdicts and stored states are identical in every pair. On the nine products with at least 8,000 stored states, TChecker is 17% to 28% faster with reuse (median) and uses up to 32% less memory on the largest; on the smallest the difference is within noise. On the benchmark requirements whose negated automaton loses a clock and whose product TChecker completes (F11–F13, R2, R3), the ratio is 0.81 to 0.90 with equal states (Supplement Section 3.6). The families are constructed, one machine and one checker were used, and UPPAAL was not measured.

9.4. Matched comparison of the two export policies

Holding the optimized automaton fixed on R2–R6, all-copy export adds no explicit edges beyond the optimized relation and no diagonal guards, while maximal-delay export adds edges in each case and between 2 and 3,360 diagonal guard atoms per formula (4,464 in total); location, clock and invariant counts are the same (Supplement Table 26). This is a property of this benchmark family, not a general size theorem.

The policy matters to a model checker by whether it can read the automaton. TChecker's liveness algorithms (`tck-liveness -a couvscc`) reject diagonal constraints ("unsupported diagonal constraints"). On the negated requirements the two policies give identical automata and TChecker counts; on the positive forms the maximal-delay export contains diagonal guards from R2 on, and TChecker stops with this error on each, while the all-copy export is accepted. With a system that violates the requirement on every run and requests only the first response, so that the product is empty and explored completely, TChecker stores 48 states for each of R1–R5 with the all-copy export, and 48, 375, 3,439 and 34,107 on R1–R4 with CASAAL's automaton (Supplement Table 16). This system exercises one response whatever n is; Section 9.2.4 repeats the comparison with a system that exercises all n responses.

Effect of the optimizer on the export. The optimizer does not change what TChecker explores, but in these runs it decides whether the TChecker-readable export is produced at all. The times are wall-clock times of the complete run of the tool (Supplement Table 24). On the positive forms of R2–R6, with one run each and a limit of 300 s, the all-copy export of R4 takes 44.0 s without the optimizer (625 edges) and 0.7 s with it (112 edges), and the all-copy exports of R5 and R6 do not finish in 300 s without it and finish in 5.7 and 55.5 s with it (single runs; the second time is close enough to the limit that run-to-run variation could matter). The maximal-delay export of R6 takes 50.9 s without the optimizer and 7.9 s with it (31,008 against 1,837 edges). Only the optimized all-copy export both is accepted by TChecker's liveness algorithms and finishes on all five requirements.

Which passes are not redundant. A leave-one-out ablation (scratch build of the extracted code whose round skips one pass; each variant still preserves the language) on the 25 fast benchmark configurations and the 100 random formulas (Supplement Table 28) shows which passes are not redundant on these inputs, not what each contributes alone. Removing the elimination of unsatisfiable guards has the largest effect (4,183 transitions instead of

670 on the benchmark, 3,579 instead of 2,366 on the random formulas); removing transition merging adds 25 and 329 transitions; removing guard normalization, dead-reset removal or clock merging leaves up to 21 more clocks; removing invariant propagation or forward propagation changes none of the 125 results.

9.5. Translator-independent synthetic suite

To exercise the interface without the MTL translator, we run the optimizer and export on 16 small synthetic TBAs and on parameterized families written as Rocq records. Each pass acts on the structure it targets and the passes interact: omitting forward propagation retains all seven edges of the forward cascade where the full optimizer leaves four (Supplement Table 27). On the 100 random formulas of the translation article the optimizer reduces transitions on 50 (3,980 to 2,366 in total) and increases none (Supplement Sections 10 and 5.1).

Independently generated TBAs. We generated 300 random timed Büchi graphs (6 to 12 locations, 2 to 4 clocks) and 300 specification-style monitors with redundant clocks, dead resets and duplicate edges, read them with `tba_opt` and exported each with both policies. All 1,200 runs complete. The bounded word test of Section 9.6 (20,000 words, 60 loop iterations) finds no difference between an input and its optimized automaton or its exported XML; the optimizer removes 2,046 of the 7,562 transitions, on 534 of the 600 inputs. The check `init_free` fails on 423 of the 600 inputs, for which only the existential initial valuation is tested. The inputs are ours, not a third-party corpus (Supplement Section 5.3, Table 9).

9.6. Model checking, agreement with the semantics, and what verification adds

UPPAAL. We generated 311 UPPAAL 5.0.0 models: the negated requirement in seven variants (ours with the default export, with `-n 0` and with the all-copy export; CASAAL's converted automaton plain and through our export in the same three ways) composed with request/response, parallel-component, sporadic and free systems. All 296 runs with an expected verdict return it and the stored states are identical in all variants except on F5 and F8 (ours 3 and 2 states against 4 and 4). These are small reachability checks: they show usability and equal verdicts, not a speedup (Supplement Section 9.7).

Agreement with the semantics. An independent evaluator of the pointwise MTL semantics, transcribed from the Rocq definition, together with acceptance evaluators for our UPPAAL XML and for CASAAL's DOT output, was run on random ultimately periodic timed words (22 benchmark patterns with 20,000 words each and 100 random formulas with 6,000 words each, both polarities). Our tool with both export policies and CASAAL (20 patterns) agree with the evaluator on all 6,160,000 word checks (122 inputs), and the evaluator rejects 11 of 13 deliberately wrong formulas for our tool and 7 of 7 for CASAAL. This finds no bug in either tool; it is agreement on a finite sample, and the evaluator shares its source with the proofs (Supplement Section 9.8).

Hypotheses of the shipped theorem. The shipped theorem assumes that Spot's automaton accepts the action-consistent words that satisfy the weakened clocked encoding of $\mathsf{recur}(f)$ (Section 7), and its guarantee for UPPAAL needs the initial-value check to succeed. We tested the first hypothesis by translation validation with an independent translator (ltl2ba) and two emptiness checks with Spot's tools: it succeeds on 25 of the 31 benchmark configurations and on 98 of the 100 random formulas, the others did not finish in time and none failed; this is evidence about these runs, outside the proofs. The initial-value check succeeded on F1–F14, R1–R6, N1–N5 and the case studies, and failed on 4 of the 100 random formulas, which nest lower-bounded operators such as $\Box_{\geq 1}\Diamond_{>1} r$, and on 423 of the 600 independently generated TBAs; the tool then warns that the result for clocks starting at 0 is not covered.

Seeded defects. Agreement does not show what the proofs add. We therefore injected 22 plausible defects of our choosing, one at a time, in a scratch copy of the extracted optimizer and exporter (a strict instead of non-strict test at the bound zero, a reset set not compared when two locations merge, the complement of $x < c$ taken as $x > c$) and tested each in five ways: comparison of the output with the unmutated tool's on the benchmark and 148 generated automata (T1); a test on up to 100,000 random words per input of whether the output accepts the words of its input (T2); both on 3,000 random automata (T3); an exhaustive comparison on every lasso word with a prefix of at most three and a loop of one to three positions (T4); and a structure-directed fuzz that gives guarded transitions a twin with the complemented guard, written after M12 escaped (T5). Eighteen mutants change the language of some automaton, and T2 finds 13 of them; five (M01, M10, M12, M16, M17) pass T1 and T2. T3 and T4 find all of them but M12,

which only T5, a generator aimed at the pass it breaks, finds. The other four (M05, M08, M21, M22) show no language change; M05 is a different composition of verified passes, and the other three are argued, not proved, to preserve the language.

The word test is a weak baseline: it misses even a single dropped transition in 36% of our controls (354 of 553 detected), and we tried no exact language-equivalence check. We applied the same mistakes to the Rocq definitions in a scratch copy of the development. It rejects all 22 transcribed mutants within 1.8 to 9.1 seconds, each at a named lemma (Supplement Table 23), including the five that T1 and T2 missed and the four that do not change the language: a rejection means that the mutated definition no longer matches the specification the lemmas are stated against, not that its language differs. The transcriptions are ours, and a mutation of implementation and specification together would pass. The claim is therefore bounded: for these defects in this code, the proofs reject all 22 without a test designed for each, while T1 and T2 missed five of the 18 language-changing ones. It does not show a defect in CASAAL, which agreed with the evaluator throughout (Supplement Section 9.9).

9.7. Limitations

TChecker states and UPPAAL verdicts are measured on small products: on the negated products, R4–R7 and N5–N6 did not finish for TChecker within 300 s, and for R8, N7 and N8 no product could be generated. CASAAL's sizes and state counts depend on our importer, which is outside the proofs. The pruning explains the large counts on the positive-form workload, which is constructed and covers one family; for that family the importer neither splits a location nor drops an edge (CASAAL prints no acceptance mark), so the overlap is in CASAAL's printed automaton, but the importer's reading of the labels as mutually exclusive letters and its expansion into one transition per disjunct are ours. Pruning is weak on generic graphs, and the timing gain of clock reuse is measured on constructed families with one checker. The mutation study uses 22 hand-chosen defects, unverified test oracles and author-made Rocq transcriptions; we kept no log of real bugs that the proofs exposed. The theorems cover words that are infinite, time-divergent and have one event per position; the UPPAAL checks are reachability of an accepting sink and no theorem relates the exported automaton to the semantics of UPPAAL or TChecker. The slowdown on deep nests rests on one profile of N7, and we built no alternative front end.

10. Related Work

Table 12 in the supplement compares the scope of the representative prior work.

Prior work already contains several of the algorithmic ideas used here. Daws and Yovine combine active-clock detection with detection of equal clocks [11], and Herrera, Westphal, and Podelski extend equal-clock reduction to quasi-equal clocks [12]. Removing inactive clocks and merging equal clocks are not new; we prove our versions as timed-word language equivalences. Guha, Narayan, and Arun-Kumar construct a timed-bisimilar automaton with the fewest possible clocks [13], and Yalcinkaya and Aydin Gol give a position-based reduction [14]; neither describes a mechanization. Saeedloei and Kluźniak study clock allocation in timed automata by liveness, interference graphs and colouring [8], with optimal allocation in polynomial time for a subclass; as we read it, their treatment is on paper and for restricted classes of automata. Our clock reuse (Section 4.2) uses the same ingredients and validates its certificate as for register allocation in CompCert [6]; we claim no new allocation method, only a Rocq-checked guarantee for Büchi automata with existential initial values and a heuristic colouring (DSATUR) that need not be optimal.

Badban, Leue, and Smaus derive control-location invariants and prune idle transitions [15], for predicate abstraction rather than under a Büchi-language contract; they note that UPPAAL admits no disjunctive location invariants and mention location splitting as a direction without giving the construction. De Wulf, Doyen, and Raskin copy locations in an encoding of the almost-ASAP semantics [16], and Bodeveix and Filali informally propose a one-copy-per-disjunct split for two disjuncts, announcing a PVS proof [17]. We make no priority claim for location splitting; we prove two split policies for our formal interface.

Mechanized optimization of automata exists for untimed Büchi automata: Schimpf and Smaus formalize bisimulation reduction and α-ball reduction in Isabelle/HOL with a verified implementation [18], and simulation-based reductions of nondeterministic Büchi automata remove transitions and merge states when one simulates another [7]. These automata have no clocks, guards or invariants. Our pruning (Section 4.1) differs from both: it is a relation on locations, with two clock sets per location (equal values on *live*, smaller or equal on *lle* for upper-bound guards), so the dominating and the dominated transition may reset different clocks, and it is checked by local conditions, not

computed as a timed simulation on configurations; we know of no mechanized timed counterpart in the work cited here.

Mechanized timed-automata work verifies checking or certification, not transformations: Wimmer and Lammich refine reachability checking to an executable Isabelle/HOL model checker [19, 20], and Wimmer, Herbreteau, and van de Pol verify a certifier for emptiness certificates of timed Büchi automata [21].

Translators from real-time logics are the closest tools to the whole pipeline. CASAAL [5] implements the construction of Li et al. [10] for $\mathrm{MTL}_{0,\infty}$, the pointwise semantics we use, with Boolean transition labels and transition-based acceptance; we compare with it in size, generation time and stored states (Sections 9.2.1–9.2.3). MightyL [22] translates MITL through very-weak one-clock alternating automata; to our knowledge it does not target the pointwise $\mathrm{MTL}_{0,\infty}$ semantics, so we did not run it. As far as we could find, none of these translators has a machine-checked correctness proof. Our chain adds no new translation construction; its contribution is mechanization and composition (Sections 7.1 and 4), supported by the seeded-defect study of Section 9.6.

11. Threats to Validity

Internal validity. The theorems concern the Rocq model of automata; the extracted OCaml functions are the verified ones, but the surrounding tool is not. The trusted base is: Spot and the reader of its output; the realization of reals by floating point and the extraction mechanism; the parser, the iteration driver, the printers and the UPPAAL XML writer; the importer used for CASAAL's automata; the OCaml and Rocq toolchains; UPPAAL and TChecker; the check `init_free`; and the project axiom `LTL_TO_BUCHI_CORRECT`, which only convenience theorems use. We reduce this risk with a second printer, a reader check (`-check`) and independent exploration of the case-study models, but these are tests, not proofs. The certificate producers of the checked passes are unverified by design: a wrong certificate is rejected, so it can cost optimization but not correctness.

Construct validity. CASAAL implements the semantics of Li et al. with integer bounds and mutually exclusive events, and it reports transition-based acceptance. We compare only configurations whose CASAAL encoding is exact in that setting and convert its acceptance to locations; the importer is our choice, and a different encoding could give different counts. Generation

times compare a Windows executable with our tool under WSL 2, so they are not precise ratios; differences below about 0.3 s are of the order of the process start.

External validity. The requirement patterns are a small corpus drawn from the literature and from parameterized families, with small system models. Results do not predict behavior on industrial specifications, we make no speedup claim from optimization, and the timing of clock reuse comes from constructed families and five benchmark requirements on one machine and one checker.

Conclusion validity. Every number is produced by scripts in the archive from raw tool output; the comparisons are counts over a fixed list of configurations, with no significance tests, and times are medians of three to five runs without spread (single runs for export scalability and N8).

12. Conclusion

We presented a Rocq-verified optimization and export pipeline for timed Büchi automata: local passes whose proofs follow the rewrite, two passes (removal of dominated transitions, reuse of clocks) whose analyses are untrusted and whose certificates a verified checker validates, a verified export with conjunctive guards and invariants, and the composition with a verified MTL translation into a modular chain whose stages are stated against interfaces. Each pass preserves the timed-word language of every input TBA, the optimizer for every number of rounds and the checked passes for every certificate.

On the negated requirements that a model checker receives, our automata have the same size as CASAAL's, after our own conversion of its acceptance, wherever both tools finish, except on three small patterns where ours are smaller; the automata are isomorphic on only 11 of 27 patterns. TChecker stores 204,564 states on ours and 204,819 on CASAAL's over the 18 products every variant completed. On the positive form of the bounded responses, composed with a constructed system that violates the requirement, CASAAL's counts grow by factors of 19, 53 and 37 on three consecutive responses (1,736,096 states on R4 against 114 for ours). The verified optimizer and export do not remove this gap, but the checked pruning does after the splitting of transitions by cells of letters: TChecker then stores exactly our counts on CASAAL's automaton for R2–R5, so the extra states are explored through dominated transitions. The optimizer shrinks the automata

without changing what the checkers explore, though in single runs it decides whether the all-copy export of R5 and R6 finishes. Pruning shrinks the positive forms of our own translator (1,504 to 1,025 transitions) without changing the stored states, and is weak on generic graphs (24 of 3,054 transitions removed). Clock reuse removes 47 of 201 clocks and makes TChecker 17% to 28% faster on constructed products. Generation is slower than CASAAL's on nested requirements, where Spot dominates in the one profile we made.

What the verified chain adds is assurance over all inputs, which a random-word test can miss: of 22 defects of our choosing in the extracted code, 18 change the language, a random-word test misses five of them, and the corresponding mistakes in the Rocq definitions are all rejected within seconds at named lemmas. The transcriptions are ours and the test is a weak baseline, so this shows the sensitivity of the proofs to edits, not defect detection.

A direct MTL-to-TBA construction in the style of CASAAL could replace the front end under the same theorems and would remove the trust in Spot; we have not written one. Future work also includes certificates that merge locations, measurements on UPPAAL, and industrial corpora.

Data and artifact availability

The Rocq development, the OCaml tool `mtl2tba`, the evaluation scripts, the inputs and the raw results are archived on Zenodo and developed in a public repository, `https://github.com/efares1/mtl-tba-verified-export` (Zenodo DOI: 10.5281/zenodo.24139280), under the MIT license, and contain the proof sources, the extracted implementation, the benchmark tables and raw data, the reproduction scripts and the UPPAAL case-study models; theorem names are given in Supplement Table 13 and Section 7.

CRediT authorship contribution statement

Elie Fares: Conceptualization, Methodology, Software, Formal analysis, Investigation, Writing – original draft. **Jean-Paul Bodeveix**: Conceptualization, Methodology, Formal analysis, Investigation, Validation, Writing – review & editing. **Azar Salami**: Formal analysis, Validation, Writing – review & editing. **Hussain Al-Aqrabi**: Validation, Writing – review & editing.

Declaration of generative AI in the writing process

During the preparation of this work the authors used Claude (Anthropic) to improve the text, to generate evaluation and checking scripts, and to assist with parts of the Rocq sources and of the tool. The authors reviewed and verified all content, checked all proofs with Rocq, and take full responsibility for the publication.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] G. Behrmann, A. David, K. G. Larsen, A tutorial on UPPAAL, in: Formal Methods for the Design of Real-Time Systems, Vol. 3185 of Lecture Notes in Computer Science, Springer, 2004, pp. 200–236. doi:10.1007/978-3-540-30080-9_7.

[2] F. Herbreteau, G. Point, TChecker: model checker for timed automata, https://github.com/ticktac-project/tchecker, version 0.8 (2025).

[3] E. Fares, J.-P. Bodeveix, H. Al-Aqrabi, A. Salami, A mechanically checked translation of $\mathrm{MTL}_{0,\infty}$ into timed Büchi automata via clocked LTL, Manuscript submitted for publication, rocq development at https://doi.org/10.5281/zenodo.23916521 (2026).

[4] A. Duret-Lutz, A. Lewkowicz, A. Fauchille, T. Michaud, E. Renault, L. Xu, Spot 2.0 — a framework for LTL and ω-automata manipulation, in: Proceedings of the 14th International Symposium on Automated Technology for Verification and Analysis (ATVA'16), Vol. 9938 of Lecture Notes in Computer Science, Springer, 2016, pp. 122–129.

[5] G. Li, CASAAL: A tool for translating $\mathrm{MTL}_{0,\infty}$ into timed Büchi automata, Online, available at the CASAAL tool page (2017). URL https://lcs.ios.ac.cn/~ligy/tools/CASAAL/

[6] S. Rideau, X. Leroy, Validating register allocation and spilling, in: Compiler Construction (CC 2010), Vol. 6011 of Lecture Notes in Computer Science, Springer, 2010, pp. 224–243.

[7] L. Clemente, R. Mayr, Efficient reduction of nondeterministic automata with application to language inclusion testing, Logical Methods in Computer Science 15 (1) (2019).

[8] N. Saeedloei, F. Kluźniak, Clock allocation in timed automata and graph colouring, in: Proceedings of the 21st International Conference on Hybrid Systems: Computation and Control (HSCC), 2018, pp. 71–80. doi:10.1145/3178126.3178138.

[9] D. Brélaz, New methods to color the vertices of a graph, Communications of the ACM 22 (4) (1979) 251–256.

[10] G. Li, P. G. Jensen, K. G. Larsen, A. Legay, D. B. Poulsen, Practical controller synthesis for $\mathrm{MTL}_{0,\infty}$, in: Proceedings of the 24th ACM SIGSOFT International SPIN Symposium on Model Checking of Software, SPIN 2017, Association for Computing Machinery, New York, NY, USA, 2017, pp. 102–111. doi:10.1145/3092282.3092303.
URL https://doi.org/10.1145/3092282.3092303

[11] C. Daws, S. Yovine, Reducing the number of clock variables of timed automata, in: Proceedings of the 17th IEEE Real-Time Systems Symposium, IEEE Computer Society, 1996, pp. 73–81. doi:10.1109/REAL.1996.563702.
URL https://doi.org/10.1109/REAL.1996.563702

[12] C. Herrera, B. Westphal, A. Podelski, Quasi-equal clock reduction: More networks, more queries, in: Proc. TACAS, Vol. 8413 of Lecture Notes in Computer Science, Springer, 2014, pp. 295–309. doi:10.1007/978-3-642-54862-8_20.

[13] S. Guha, C. Narayan, S. Arun-Kumar, Reducing clocks in timed automata while preserving bisimulation, in: CONCUR 2014—Concurrency Theory, Vol. 8704 of Lecture Notes in Computer Science, Springer, 2014, pp. 527–543. doi:10.1007/978-3-662-44584-6_36.

[14] B. Yalcinkaya, E. A. Gol, Clock reduction in timed automata while preserving design parameters, in: Proceedings of the 7th International Workshop on Formal Methods in Software Engineering, 2019, pp. 11–20. doi:10.1109/FormaliSE.2019.00010.

[15] B. Badban, S. Leue, J.-G. Smaus, Automated invariant generation for the verification of real-time systems, in: A. Voronkov, L. Kovacs, N. Bjorner (Eds.), WING 2010: Workshop on Invariant Generation 2010, Vol. 1 of EPiC Series in Computing, EasyChair, 2012, pp. 44–58. doi:10.29007/npn7.
URL https://doi.org/10.29007/npn7

[16] M. D. Wulf, L. Doyen, J.-F. Raskin, Systematic implementation of real-time models, in: FM 2005: Formal Methods, Vol. 3582 of Lecture Notes in Computer Science, Springer, 2005, pp. 139–156. doi:10.1007/11526841_11.

[17] J.-P. Bodeveix, M. Filali, About disjunctive invariants in UPPAAL, in: 15th Nordic Workshop on Programming Theory (NWPT), Turku, Finland, 2003, workshop presentation, 29–31 October 2003.
URL http://www.abo.fi/~Kaisa.Sere/15NWPT.html

[18] A. Schimpf, J.-G. Smaus, Büchi automata optimisations formalised in Isabelle/HOL, in: Logic and Its Applications (ICLA 2015), Vol. 8923 of Lecture Notes in Computer Science, Springer, 2015, pp. 158–169.

[19] S. Wimmer, P. Lammich, Verified model checking of timed automata, in: Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2018), Lecture Notes in Computer Science, Springer, 2018, pp. 61–78.

[20] S. Wimmer, Munta: A verified model checker for timed automata, in: Formal Modeling and Analysis of Timed Systems (FORMATS 2019), Lecture Notes in Computer Science, Springer, 2019, pp. 236–243. doi:10.1007/978-3-030-29662-9_14.

[21] S. Wimmer, F. Herbreteau, J. van de Pol, Certifying emptiness of timed Büchi automata, in: Formal Modeling and Analysis of Timed Systems (FORMATS 2020), Vol. 12288 of Lecture Notes in Computer Science, Springer, 2020, pp. 58–75. doi:10.1007/978-3-030-57628-8_4.

[22] T. Brihaye, G. Geeraerts, H.-M. Ho, B. Monmege, MightyL: A compositional translation from MITL to timed automata, in: R. Majumdar, V. Kunčak (Eds.), Computer Aided Verification, Springer International Publishing, Cham, 2017, pp. 421–440.